

Backtracking: put down recursion tree
as soon as we find
impossibility

Given a string, print all those permutations which
don't contain "AB" as a substring.

Ip: str = "ABC"

O/P: ACB, BAC, BCA, CBA, ~~ABC~~, ~~CAB~~

Naive Solution:

void permute (String str, int l, int n)

{ if (l == n)

{ if (str.find("AB") == string::npos)

{ print(str);

return;

}

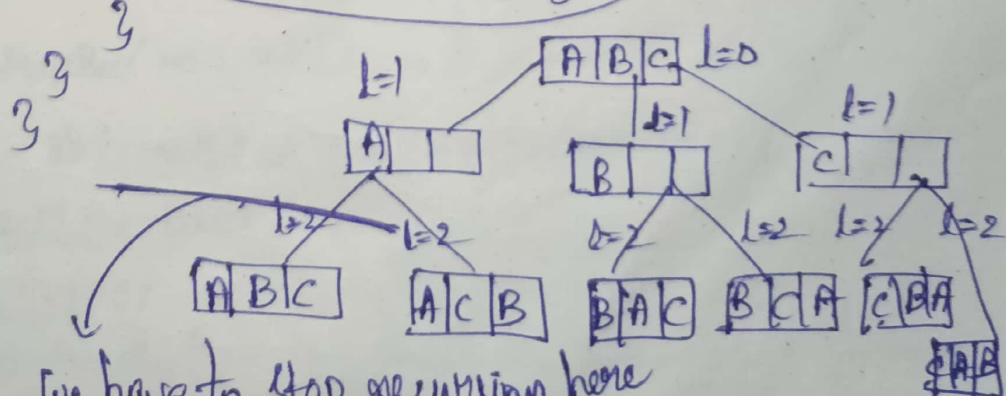
else { for (int i = l; i <= n; i++)

{ swap(str[l], str[i]); permute(str, l+1, n);

swap(str[l], str[i]);

}

T.C = $n \times n!$



We have to stop recursion here
if there is a chance to get "AB" as a substring.

Backtracking approach:

bool isSafe(string str, int l, int i, int n)

2 if (l != 0 && str[l-1] == 'A' && str[i] == 'B')
return false;

if (n == (l+1) && (str[i] == 'A' && str[l] == 'B') ||
(str[i] == 'A' && str[n] == 'B'))

return false;

return true;

3

(BA)

condition
checking

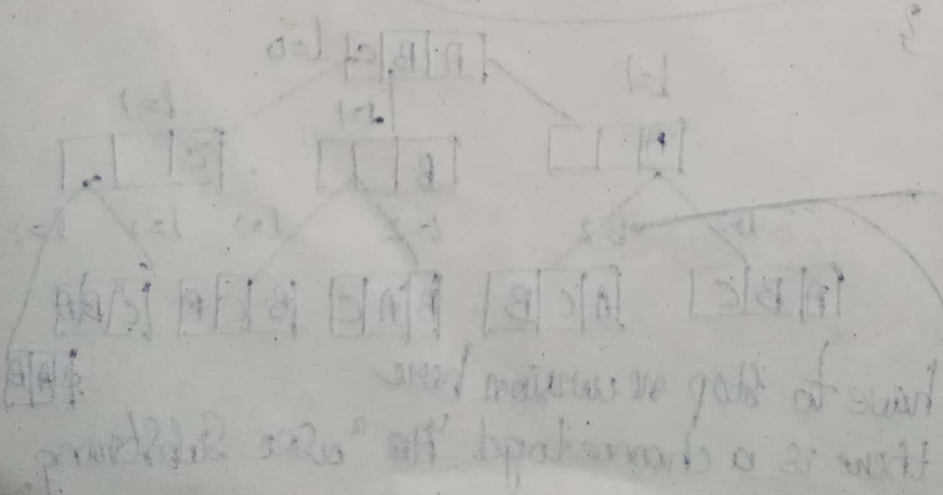
Backtracking T.C = $n! \times n$

$n!$

T.C = $n! \times n$

(+1) : (1, 2, 3, 4, 5, 6, 7, 8, 9, 10) not

(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) not
(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) not
(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) not
(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) not



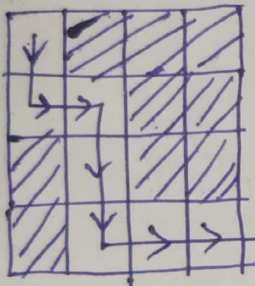
Rat in a Maze

IP: maze[][] = $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$

NOTE: only two moves allowed from (i, j)
 ① (i+1, j)
 ② (i, j+1)

OP: yes

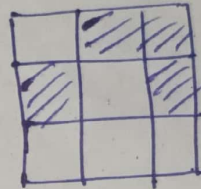
1 0 0 0
 1 1 0 0
 0 1 0 0
 0 1 1 1



IP:

$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}$

OP: No



1) rat is always at top left
 2) cheese is always at bottom right

```
boolean maze[N][N], sol[N][N];
boolean solveMaze()
{
    if (solveMazeRec(0,0) == false)
        return false;
    else
    {
        printMatrix(sol);
        return true;
    }
}
```

```
boolean solveMazeRec(int i, int j)
```

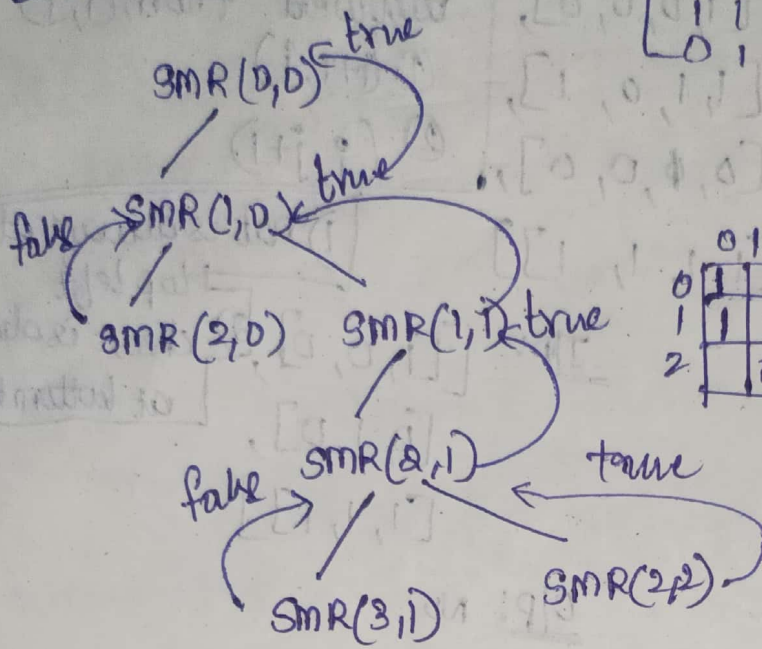
```
{
    if (i == N-1 && j == N-1) { sol[i][j] = 1; return true; }
    if (isSafe(i,j) == true)
    {
        sol[i][j] = 1;
        if (solveMazeRec(i+1,j) == true) return true;
        else if (solveMazeRec(i,j+1) == true) return true;
    }
    sol[i][j] = 0;
    return false;
}
```

Backtracking Solution:

```
boolean isSafe(int i, int j)
{
    return (i < N && j < N &&
           maze[i][j] == 1);
}
```

SMR $\Rightarrow$ solve more Rec L)

$$\begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$



	0	1	2
0	1		
1	1	1	
2		1	1

modular arithmetic

(a, b) mod m = (a mod m, b mod m)

if (a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)

(a, b) mod m = (a mod m, b mod m)



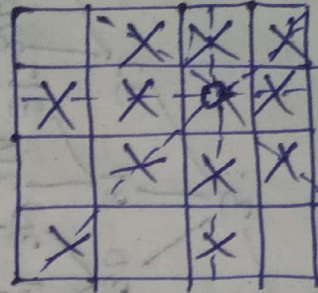
N Queen Problem

NOTE: No two queens attack each other

I/P: $N=4$

O/P: yes

0	0	1	0
1	0	0	0
0	0	0	1
0	1	0	0



⊗ ⇒ In these marks
no queens can be
placed

I/P: $N=3$

O/P: NO

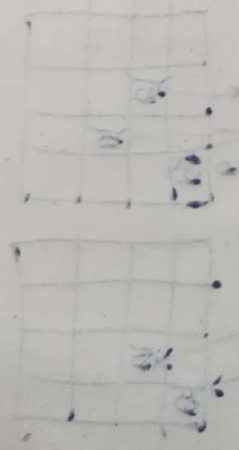
I/P: $N=5$
yes

O/P:

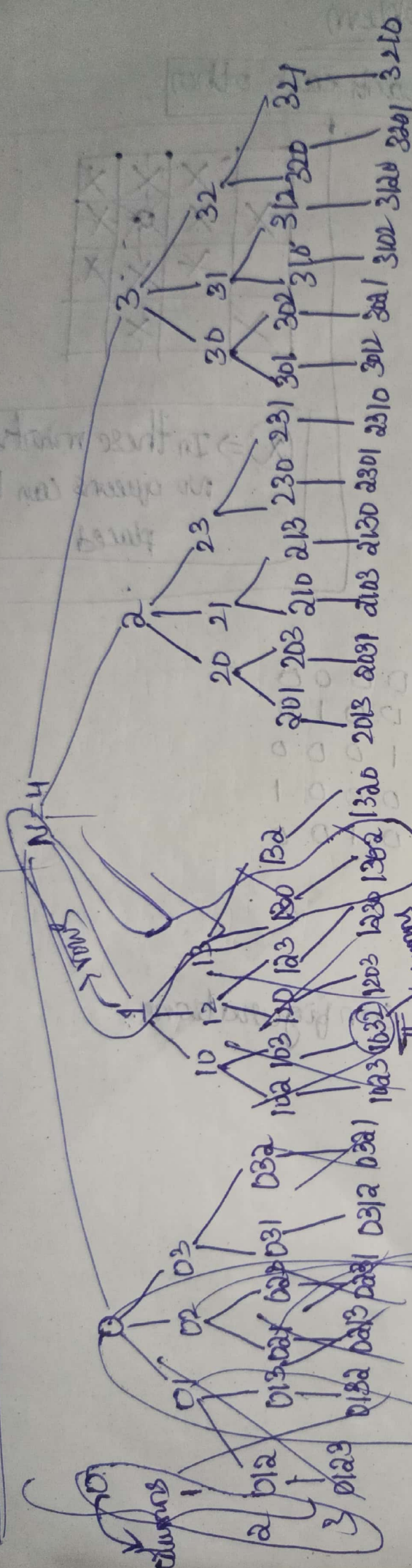
1	0	0	0	0
0	0	0	1	0
0	1	0	0	0
0	0	0	0	1
0	0	1	0	0

Super Naive Solution

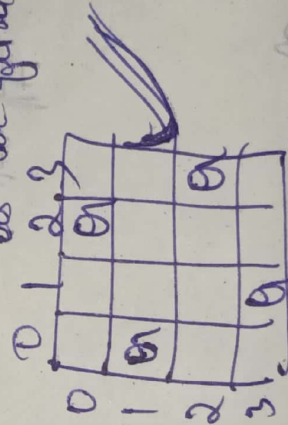
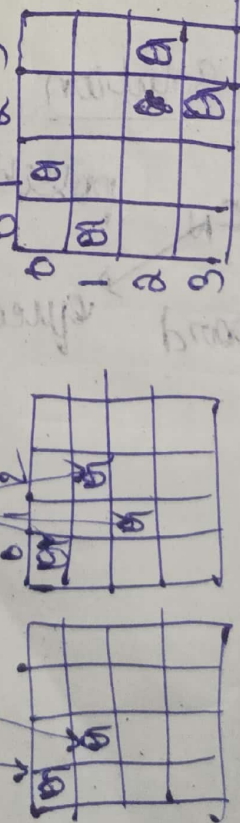
Generate N^2 possible configurations.
↙ ↘
checkboard queens



Naive Solution: Generate all permutations of row Number



Backtracking: cut down recursion tree as soon as we find infeasibility.




```
bool board[N][N];
```

```
bool solve()
```

```
{ if (solveRec(0) == false)  
    return false;
```

```
else
```

```
{ printMatrix(board);  
  return true;  
}
```

```
bool solveRec(int col)
```

```
{ if (col == N) return true;
```

```
for (int i = 0; i < N; i++)
```

```
{ if (isSafe(i, col)
```

```
{ board[i][col] = 1;
```

```
if (solveRec(col+1))  
    return true;
```

```
board[i][col] = 0;
```

```
}
```

```
return false;
```

```
}
```

```
bool isSafe(int row, int col)
```

```
{ for (int i = 0; i < col; i++)
```

```
if (board[row][i])  
    return false;
```

```
for (int i = row; j = col; i >= 0 &&  
j >= 0; i--, j--)
```

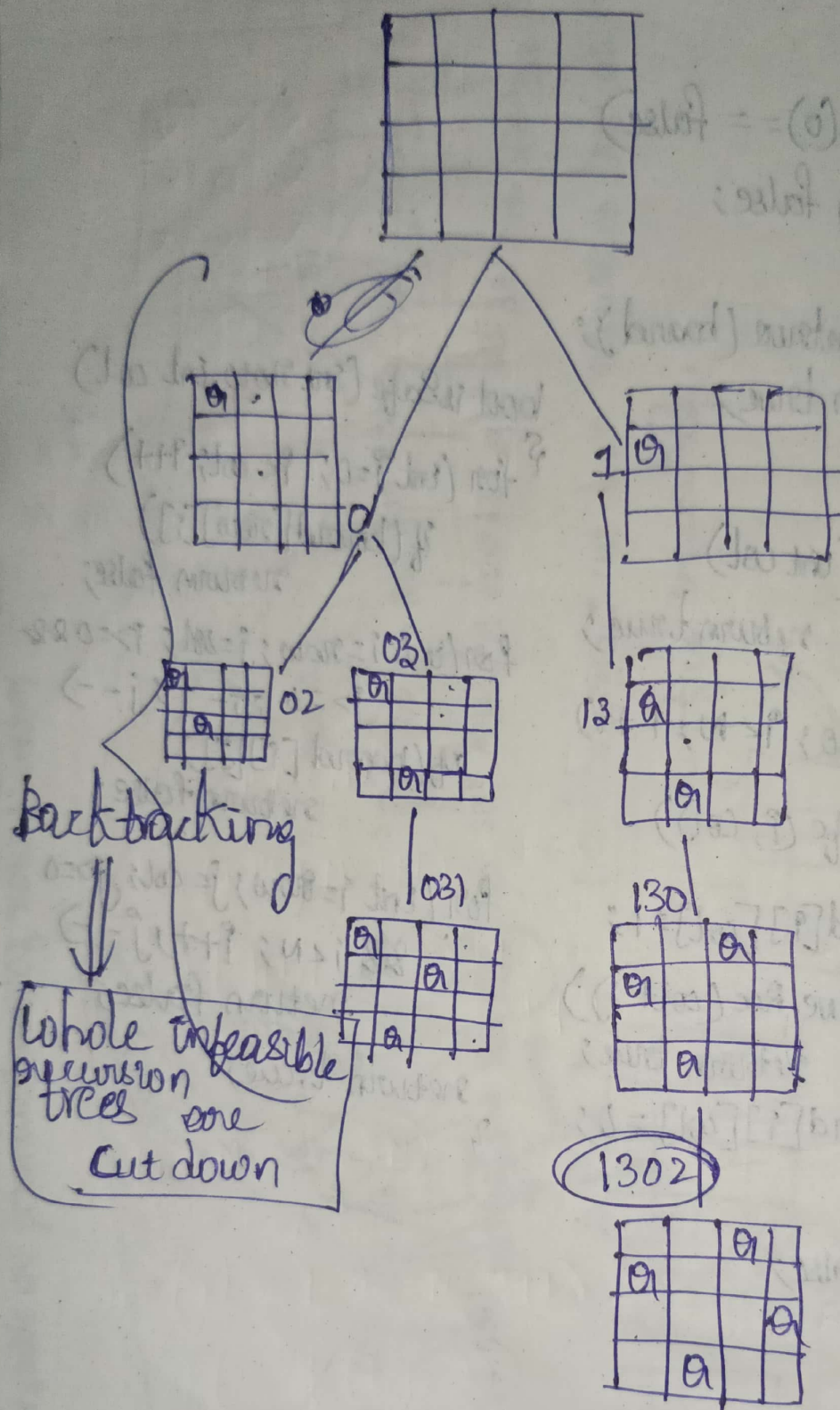
```
if (board[i][j])  
    return false;
```

```
for (int i = row; j = col; j >= 0  
&& i < N; i++, j--)
```

```
return false;
```

```
return true;
```

```
}
```



Sudoku problem

Rules to fill Empty cells

- 1) every no. in every row should be diff.
- 2) every No. in every col should be diff.
- 3) every No. in Subgrid must be diff.



In 4x4 grid there are 4 $\Rightarrow$ 2x2 Subgrids

I/P: grid[][] = $\begin{bmatrix} 1, 0, 3, 0 \\ 0, 0, 2, 1 \\ 0, 1, 0, 2 \\ 2, 4, 0, 0 \end{bmatrix}$ $0 \Rightarrow$ empty cells

O/P: grid[][] = $\begin{bmatrix} 1, 2, 3, 4 \\ 4, 3, 2, 1 \\ 3, 1, 4, 2 \\ 2, 4, 1, 3 \end{bmatrix}$

Naive Solution:

Try all X^N configuration for given X empty spaces and $N \times N$ grid.

Efficient Solution: Use Backtracking.

Begin with an empty cell and a number, & try filling rest of the cells.

1	2	3	4
4	3	2	1
3	1	4	2
2	4	1	3

$N=4, X=8$

```

bool solve()
{
    int i, j;
    for (i=0; i<N; i++)
    {
        for (j=0; j<N; j++)
        {
            if (g[i][j] == 0) // First empty cell
            {
                break;
            }
        }
        if (i == N && j == N)
            return true;
        for (int a=1; a<=N; a++)
        {
            if (isSafe(i, j, a))
            {
                g[i][j] = a;
                if (solve()) return true;
            }
            g[i][j] = 0;
        }
        return false;
    }
}

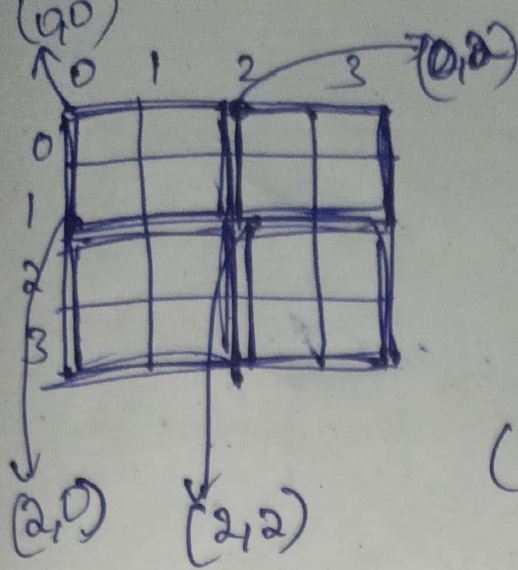
```

```

int N=4;
int g[N][N];
bool isSafe(int i, int j, int a)
{
    for (int k=0; k<N; k++)
    {
        if (g[k][j] == a || g[i][k] == a)
        {
            return false;
        }
    }
    int s = sqrt(N);
    int rs = i - i % s;
    int cs = j - j % s;
    for (int k=0; k<s; k++)
    {
        for (int j=0; j<s; j++)
        {
            if (g[i+rs][j+cs] == a)
            {
                return false;
            }
        }
    }
    return true;
}

```

row column checking



$$\begin{matrix} i & j \\ (3,1) \end{matrix}$$

$$S = 2$$

$$g_s = 3 - 3/2 = 2$$

$$C_s = 1 - 1/2 = 0$$

$$g[2+2][0+0]$$

$$g[0+2][1+0]$$

solve()

$$\rightarrow g[0][1] = 2$$

$\rightarrow$ solve()

$$\rightarrow g[1][0] = 3$$

$\rightarrow$ solve()

$$\rightarrow g[1][0] = 4$$

$\rightarrow$ solve()

$$\rightarrow g[1][1] = 3$$

$\rightarrow$ solve()

$$\rightarrow g[2][0] = 3$$

$\rightarrow$ solve()

$$\rightarrow g[3][3] = 3$$

1	2	3	4
4	3	3	2
3	1	4	2
2	4	1	3